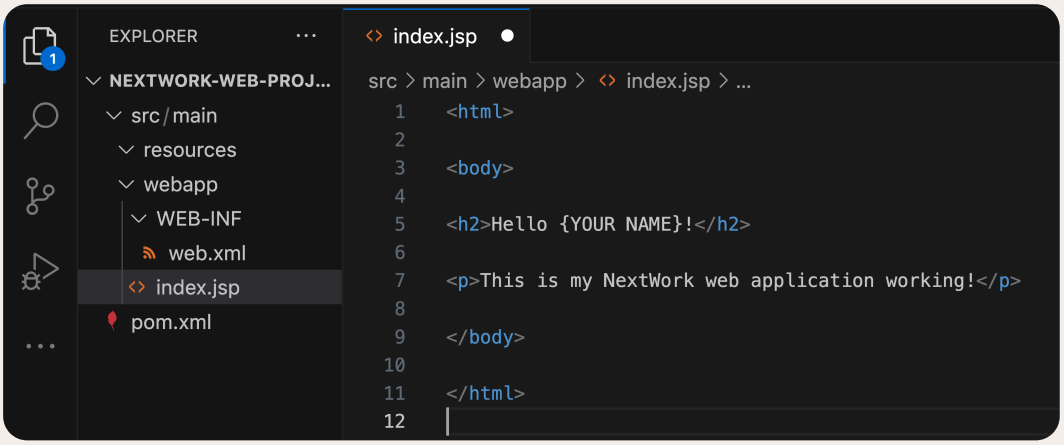
SET UP A WEB APP IN THE CLOUD

October 2026



Introducing Today's Project!

In this project, I am going to set up the foundations of a CI/CD pipeline by creating a web app from scratch. We will need to launch an EC2 instance and connect to the instance using VSCode to generate a web app inside.

This project is part one of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project next week.

I did this project to learnt how to set up a web app using EC2 instances, part of my devops journey.

Key tools and concepts

Services I used were: Amazon EC2 Key concepts I learnt include ssh connections using IDE, launching an instance, creating key pair, editing an index.jsp file, using key pairs, creating a maven java project.

Project reflection

This project took me approximately 2.5 hrs including troubleshooting time. The most challenging part was the amount of repeated connection/disconnection to the ec2 instance's project.

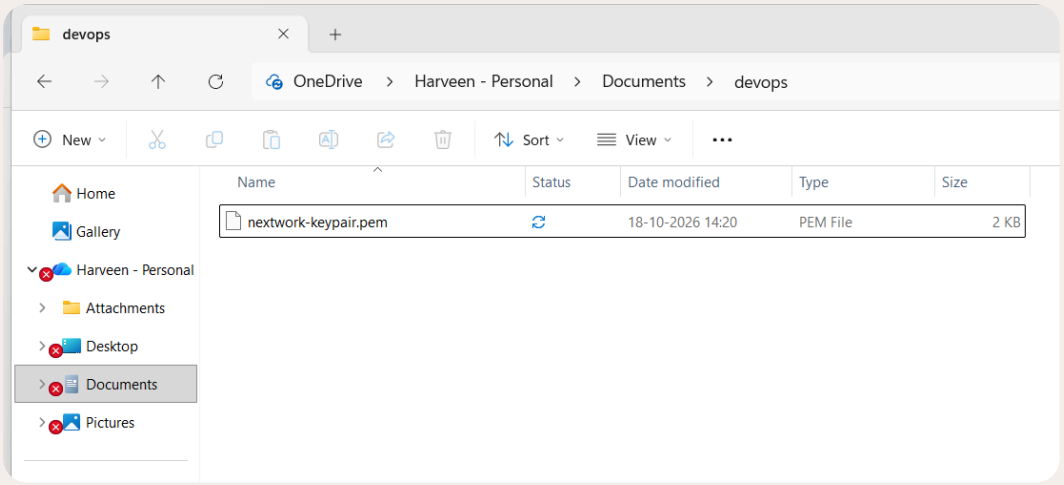
One thing I didn't expect in this project was how we would connect to the ec2 instance using the terminal and would be working on the web app that was inside the instance.

Launching an EC2 instance

What I did in this step

In this step, we r launching an ec2 instance in our aws account, we are also going to set up it's network settings which helps us make sure we can find our ec2 instance and our key pair which helps us make sure we have the permission to use the ec2 instance or access it.

I started this project by launching an EC2 instance because ec2 instances are like virtual computers in the cloud. We want our web app to live entirely on the cloud, so we are launching an ec2 instance to even develop our web app's code.



I also enabled SSH

SSH is a protocol used to make sure that only authorised users can access a remote sever like ec2 instances. It's also a type of traffic that lets us transfer data back and forth with our ec2 instance once we are connected to it, we enable ssh so we can securely connect to our ec2 instance later and we make sure that it only allows that it enables ssh traffic only from our ip address for security.

Key pairs

A key pair is a mechanism for us to get access/connect to ec2 instances we launched in aws. It's made of two halves: a public key that AWS keeps, and a private key that we download. Aws will authenticate our private key when we want to connect to our ec2 instance later, by matching it to the private key that it has.

When you use the private key, it verifies that you're the one allowed to access that specific virtual machine, keeping everything secure and just for you. We've created a key pair for the ec2 instance that we are launching.

Downloaded key pair file

Once we set up my key pair, AWS automatically downloaded the private key which was downloaded in .pem, The .pem format stands for Privacy Enhanced Mail. It started off as a way to secure emails and has become the go-to format for cryptographic keys because it works with many different types of servers e.g. EC2 instances.

Set up VS Code

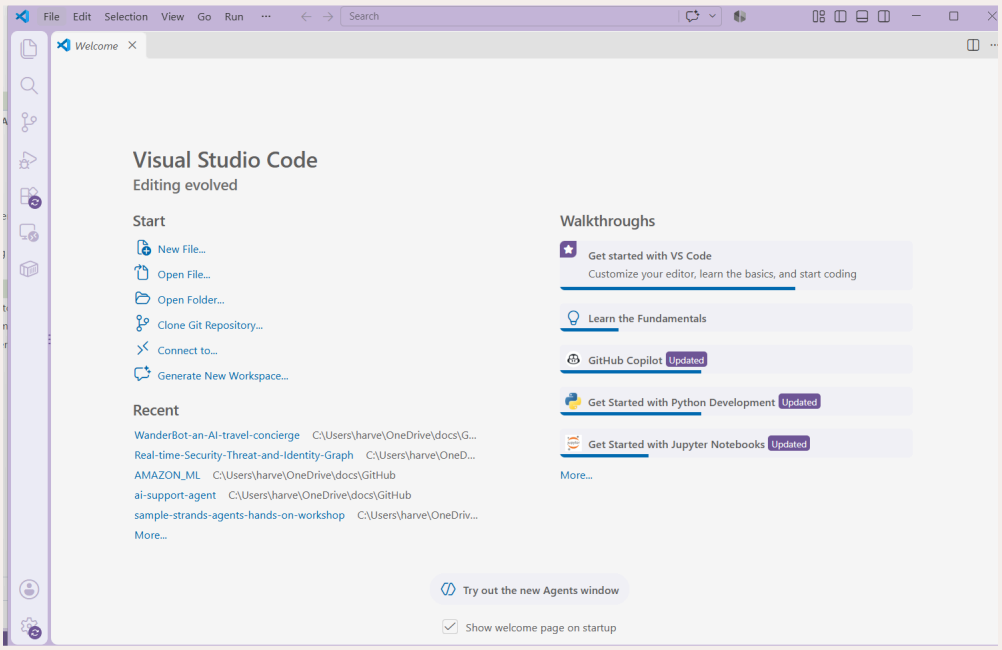
What I did in this step

In this step, I will install VSCode(write/edit code- free) and set up a terminal so we can communicate with the ec2 instance and then update our key pair's permission settings so we can use it to log in to our ec2 instance later.

What is VS Code?

VS Code is an (one of the most popular ides) IDE, software that helps you write/edit code files. It has also handy extensions that lets us direclty connect to ec2 instances so that we can write/edit code that will live inside the instance., directly.

I installed VS Code to write and edit our web app's code. We will also be going to make the most out of the ability to connect directly with an ec2 instance using the termianl it provides.

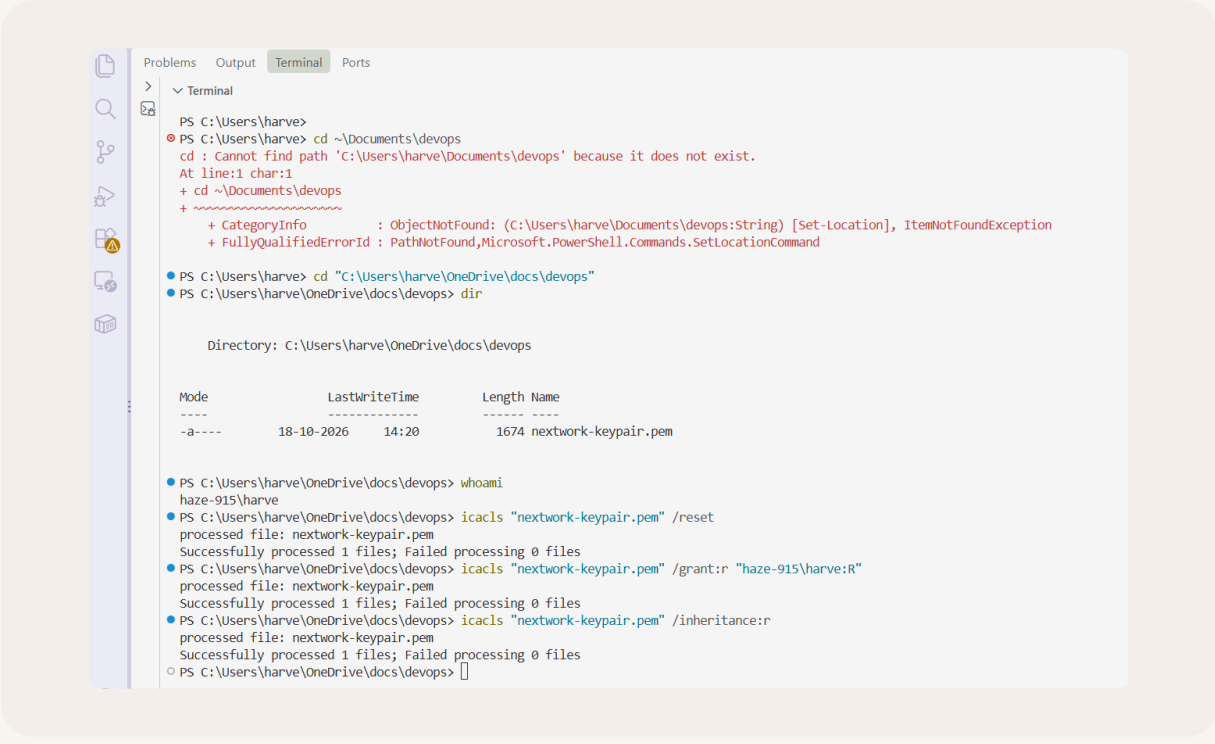


My first terminal commands

A terminal is a place for us to send commands to our computer i.e. tell it what we want to do, difference bw termianl and our usual clik-drag interface, we send text commands in terminal. The first command I ran for this project is cd "C:\Users\harve\OneDrive\docs\devops", which navigates the terminal to the devops folder we created to store our private key file that we downloaded.

Updating file permissions

I also updated my private key's permissions by running the commands - icacls "nextwork-keypair.pem" /reset : this command removes all the exisitng permissions so we can start from a clean state, icacls "nextwork-keypair.pem" /grant:r "haze-915\harve:R" : gives only my user account read access to the file, icacls "nextwork-keypair.pem" /inheritance:r : stops the file from inheriting permissions from its parent folder, keeping it locked down. We do this so we can connect to the ec2 instance.



SSH connection to EC2 instance

What I did in this step

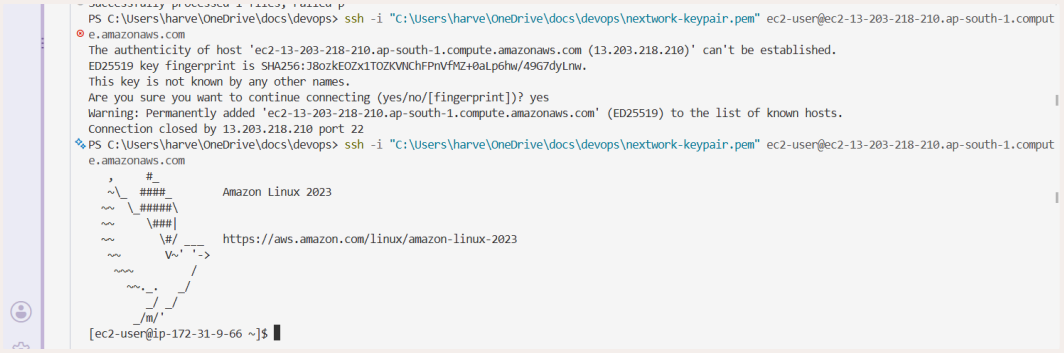
over the terminal, In this step, I will be going to try to connect to ec2 instance, we can start setting our web app code inside.

Connecting to EC2

To connect to my EC2 instance, I ran the command ssh -i path-to-private-key ec2-user@ipv4address-of-ec2-instance, this commands sets up an ssh connection direclty between our local computer and the instance,

This command required an IPv4 address

A server's IPV4 DNS is public address that identifies where the server lives in the cloud. In our case, in ec2 instances ipv4 dns is useful information to give to our local computer, it tells the local computer where to find the ec2 instance that we launched.



Maven & Java

What I did in this step

In this step, I will install apache maven in java corretto 8 in our ec2 instance, we r doing this because both these tools are important in setting up a java web app from scratch.

Why I'm using Maven

Apache Maven is a tool that helps us with creating and organising java projects. It has a lot of usecases like being a package manager, downloading external pieces of ccode, aside from being a package manager, tool that uses archetypes which r like templates(lay down the foundation for different types of projects) for spinning up projects like web apps.

Maven is required in this project because we need to use its ability to spin up web apps uisng archetypes, set up our java web app using the web app archetype.

Why I'm using Java

Java is a programming language that we are using to develop our web app. We can use it to develop various applications- mobile apps, enterprise systems etc.

Java is required in this project because it lays the foundation of writing our web app code. It's like needing to know a language before we speak it. Maven also needs Java in order to work.

Create the Application

What I did in this step

In this step, I will be generating a java web application inside our ec2 instance using maven.

Creating the Java web app

I generated a Java web app using the command mvn archetype:generate -DgroupId=com.nextwork.app -DartifactId=nextwork-web-project -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false
where mvn archetype:generate command specifically tells Maven to create a new project from a template (which Maven calls an archetype). This command sets up a basic structure for your project, so you don't have to start from scratch.
-DartifactId=nextwork-web-project names your project

-DarchetypeArtifactId=maven-archetype-webapp specifies that you're creating a web application.

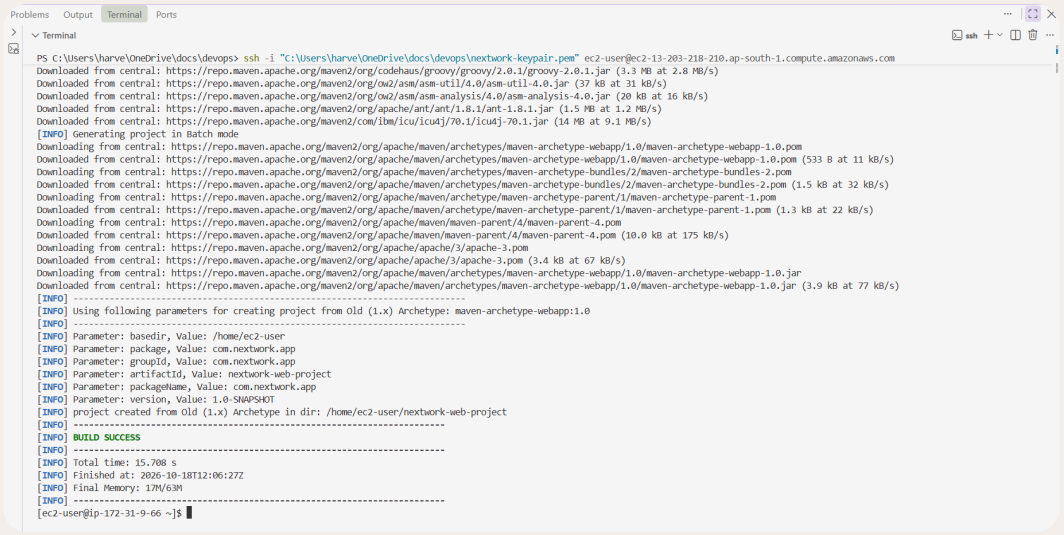
-DinteractiveMode=false runs the command without pausing for user input, so Maven will go ahead and install everything without waiting for your confirmation.

Installing Remote - SSH

I installed Remote - SSH, which is an extension within VSCode. This extension lets us connect vscode directly to our remote server, like an ec2 instance.

SSH configuration details

Configuration details required to set up a remote connection include hostname i.e ec2 instance's address and identity file(location of our private key), and the user (the user that we are logging into for our instance),


Problems
Output
Terminal
Ports
Terminal
ssh
PS C:\Users\harve\OneDrive\docs\devops> ssh -i "C:\Users\harve\OneDrive\docs\devops\nextwork-keypair.pem" ec2-user@ec2-13-203-218-210.ap-south-1.compute.amazonaws.com
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/groovy/groovy/2.0.1/groovy-2.0.1.jar (3.3 MB at 2.8 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-util/4.0/asm-util-4.0.jar (37 kB at 31 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-analysis/4.0/asm-analysis-4.0.jar (20 kB at 16 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/ant/ant/1.8.1/ant-1.8.1.jar (1.5 MB at 1.2 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/ibm/icu/icu4j/70.1/icu4j-70.1.jar (14 MB at 9.1 MB/s)
[INFO] Generating project in Batch mode
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-webapp/1.0/maven-archetype-webapp-1.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-webapp/1.0/maven-archetype-webapp-1.0.pom (533 B at 11 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-bundles/2/maven-archetype-bundles-2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-bundles/2/maven-archetype-bundles-2.pom (1.5 kB at 32 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetype/maven-archetype-parent/1/maven-archetype-parent-1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetype/maven-archetype-parent/1/maven-archetype-parent-1.pom (1.3 kB at 22 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/4/maven-parent-4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/4/maven-parent-4.pom (10.0 kB at 175 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/3/apache-3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/3/apache-3.pom (3.4 kB at 67 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-webapp/1.0/maven-archetype-webapp-1.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-webapp/1.0/maven-archetype-webapp-1.0.jar (3.9 kB at 77 kB/s)
[INFO] --
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO] --
[INFO] Parameter: basedir, Value: /home/ec2-user
[INFO] Parameter: package, Value: com.nextwork.app
[INFO] Parameter: groupId, Value: com.nextwork.app
[INFO] Parameter: artifactId, Value: nextwork-web-project
[INFO] Parameter: packageName, Value: com.nextwork.app
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/ec2-user/nextwork-web-project
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 15.708 s
[INFO] Finished at: 2026-10-18T12:06:27Z
[INFO] Final Memory: 17M/63M
[INFO] --
[ec2-user@ip-172-31-9-66 ~]$

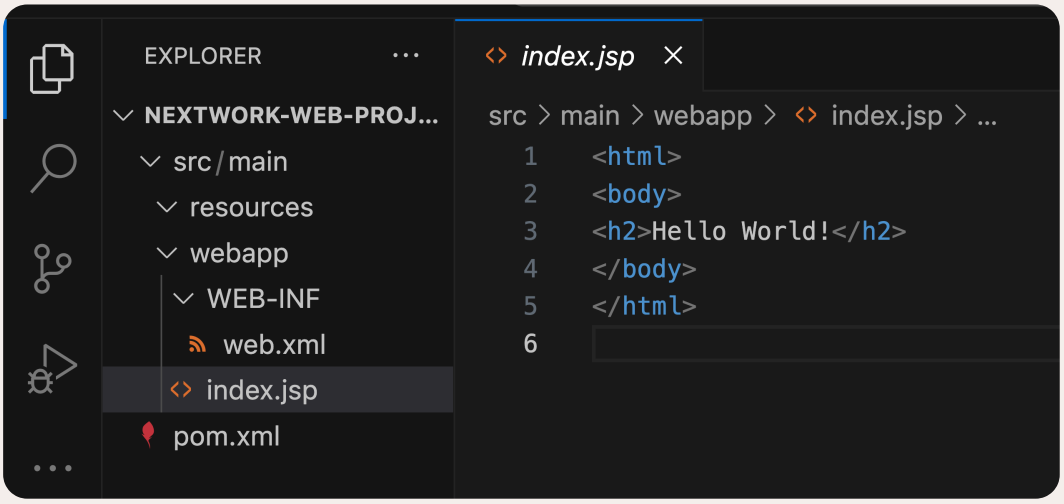
Create the Application

Exploring the project structure

Using VS Code's file explorer, I could see files and folders, i.e. the src folder that held all the source code files that define how the web app actually looks like and how works. It has two subfolders- webapp and resources, webapp where there are web app's files: .xml and .jsp. Resources are the configuration files a web app might need i.e. the connection settings to a database.

Two of the project folders created by Maven are src and webapp, which The src (source) folder holds all the source code files that define how your web app looks and works.

src is further divided into webapp, which are the web app's files e.g. HTML, CSS, JavaScript, and JSP files, and resources, which are the configuration files a web app might need e.g. connection settings to a database.



Using Remote - SSH

What I did in this step

In this step, I will connect VSCode with our ec2 instance, this is diffr from just connecting the terminal (i.e. our local computer) to our ec2 instance.

Updating the web app

The index.jsp is a file that is used in java web apps, it's similar to an HTML file as it also contains markup that is used to display web pages. However, it can also include java code which lets us generate dynamic content.

I edited index.jsp by changing the placeholder code to the code- mention "this is my NextWork web application working! using   tags. And then saved the same.

